NATURAL LANGUAGE UNDERSTANDING ASSIGNMENT - 2 REPORT

GitHub Repository Link - https://github.com/Layaa-V/M25CSA017-NLU-A2

Problem 1 - Learning Word Embeddings from IIT Jodhpur Data

The objective of this assignment was to train Word2Vec models on textual data collected from IIT Jodhpur sources and analyze the semantic structure captured by the learned embeddings.
It was comprised of 4 tasks -

- Dataset Preparation
- Model Training
- Semantic Analysis
- Visualization

Task 1- Dataset Preparation -

Data sources used in this problem are :

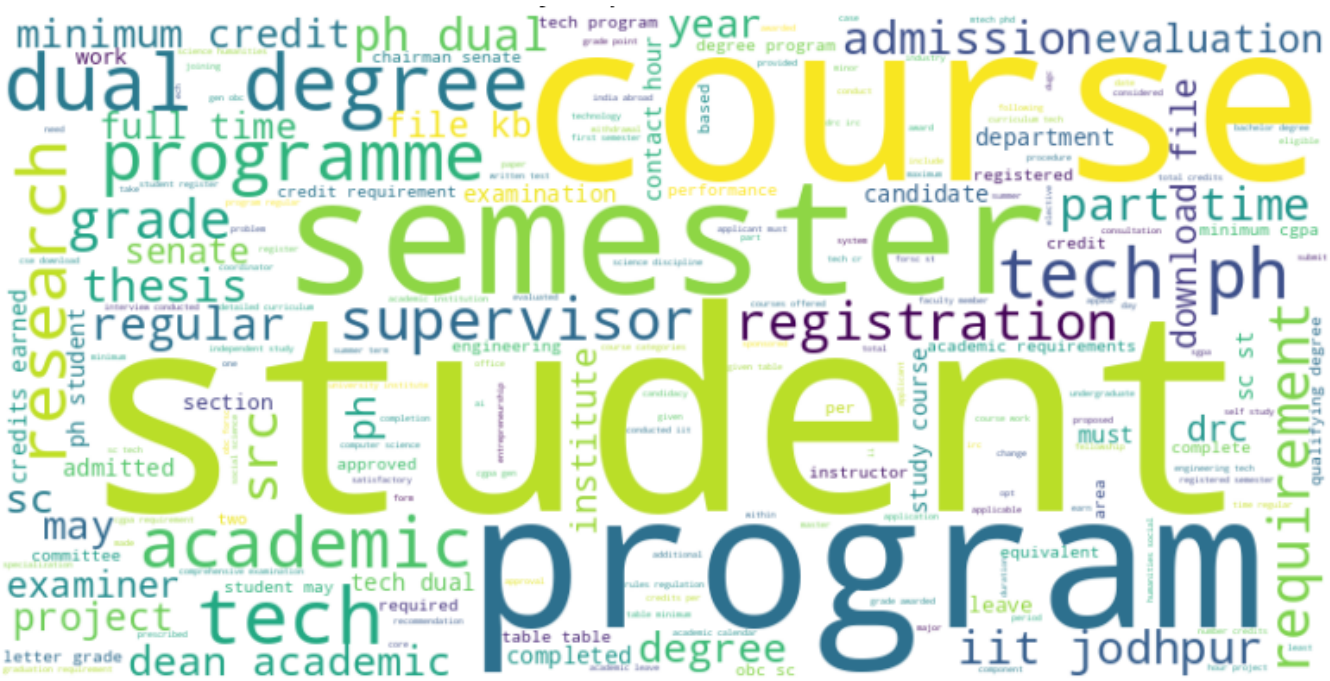
- Official IITJ Websites used to scrape data -
  "https://www.iitj.ac.in/computer-science-engineering/en/undergraduate-programs",
  "https://www.iitj.ac.in/computer-science-engineering/en/postgraduate-programs",
  "https://www.iitj.ac.in/computer-science-engineering/en/doctoral-programs"
- 3 Academic Regulation pdfs (stored in 'data_pdfs' folder in repository)

This raw data was then preprocessed (cleaning, punctuation removal, etc.), tokenized and checked for stop words.

The dataset statistics were as follows -

Total Documents: 3 URLs, 3 PDFs
Total Tokens: 14134 (as seen in the code present in GitHub repo)
Vocabulary Size: 2207 (as seen in the code present in GitHub repo)

Word Cloud Analysis -



The generated Word Cloud visually validates the dataset's academic nature. The most prominent terms include "course", "student", "program", "semester", "supervisor", "tech", “registration", “dual” and "degree".
This confirms that the cleaned corpus heavily reflects the structural and administrative language of IIT Jodhpur's academic framework.

Task 2- Model Training -

Two Word2Vec architectures, Continuous Bag of Words (CBOW) and Skip-gram, were trained from scratch. To analyze the impact of network capacity and context scope, experiments were conducted using different combinations of hyperparameters:
Embedding Dimension, Context Window Size, and Number of Negative Samples.

Model	Embed Dim	Window Size	Neg Samples	Final Loss
CBOW	50	3	3	0.5776
Skip-gram	50	3	3	0.6573
CBOW	100	5	5	0.3206
Skip-gram	100	5	5	0.6534

Observations -

- Increasing the model complexity (Dimensions: 100, Window: 5, Neg Samples: 5) resulted in a dramatic drop in the final loss for the CBOW model (from 0.5776 to 0.3206).
- The Skip-gram loss remained relatively stable across both configurations.
- This is expected behavior: CBOW predicts a target word by averaging its context, making it easier to minimize loss on a small, repetitive academic dataset. Skip-gram predicts multiple context words from a single target word, making the optimization task inherently more difficult but often yielding richer semantic representations for rare words.

Task 3 - Semantic Analysis -

The models trained with the (Dim: 100, Win: 5, Neg: 5) configuration were used to evaluate semantic capabilities using cosine similarity.

1. **Nearest Neighbors Comparison -**

   These are the nearest neighbors obtained from those 2 different methods, CBOW and Skip-gram - (as seen in the code in repo)

```
CBOW nearest neighbors
Top 5 for 'research': ['selecting', 'demand', 'co', 'highly', 'carry']
Top 5 for 'student': ['proportional', 'bog', 'selection', 'fractalcourses', 'direction']
Top 5 for 'phd': ['second', 'considerable', 'using', 'alone', 'answer']
Top 5 for 'exam': ['fellowshipsr', 'act', 'government', 'cleared', 'subsection']

Skipgram nearest neighbors
Top 5 for 'research': ['proposal', 'labs', 'compute', 'scholars', 'idrps']
Top 5 for 'student': ['perform', 'within', 'ncc', 'submit', 'shall']
Top 5 for 'phd': ['mtech', 'doctor', 'dual', 'programs', 'philosophy']
Top 5 for 'exam': ['taking', 'cer', 'assessing', 'art', 'top']
```

Observation -

We can clearly see that the Skip-gram model drastically outperforms CBOW in this task.

- Skip-gram successfully captured the semantic meaning of academic terms within the IIT Jodhpur context. For example, it correctly associates -
  - **“phd"** with “doctor”, "mtech" and “programs"
  - **"research"** with "proposal" and “scholars”

- The CBOW model's nearest neighbors appear to be mostly syntactic noise or random co-occurring words (e.g., linking "phd" to "second" or "alone"), proving that CBOW struggles to build meaningful semantic vectors on very small, specialized datasets.

2. **Analogy Experiments -**

By performing some operations, the following analogical results were obtained, one from each model.

- **undergraduate : btech :: postgraduate : ?**
  - CBOW Result: conferences (Sim: 0.416)
  - Skip-gram Result: collaboration (Sim: 0.472)

- **btech : undergraduate :: mtech : ?**
  - CBOW Result: fresh (Sim: 0.316)
  - Skip-gram Result: oneoftheidrpco (Sim: 0.328)

- **exam : grade :: thesis : ?**
  - CBOW Result: capabilities (Sim: 0.351)
  - Skip-gram Result: request (Sim: 0.331)

Observation -

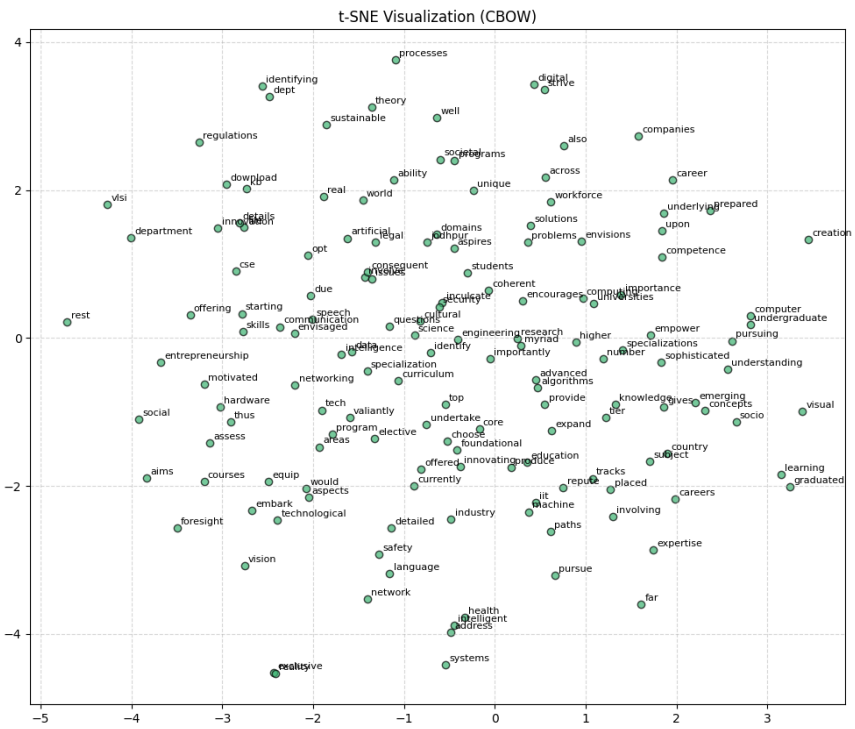
- We can see that the results are not semantically meaningful. Neither model was able to correctly solve the analogies. This problem arises due to limitation of our corpus size. To map the relationship between two similarly used (semantically similar) words (eg “undergraduate” and “btech”) the model needs to see those words used in diverse contexts hundreds of times.
- Because our dataset is limited to a few IIT Jodhpur web pages and PDFs, the vector space is too sparse and the models simply guess the mathematically closest point, resulting in words like “oneoftheidrpco" that make no valid sense.

Task 4 - Visualization -

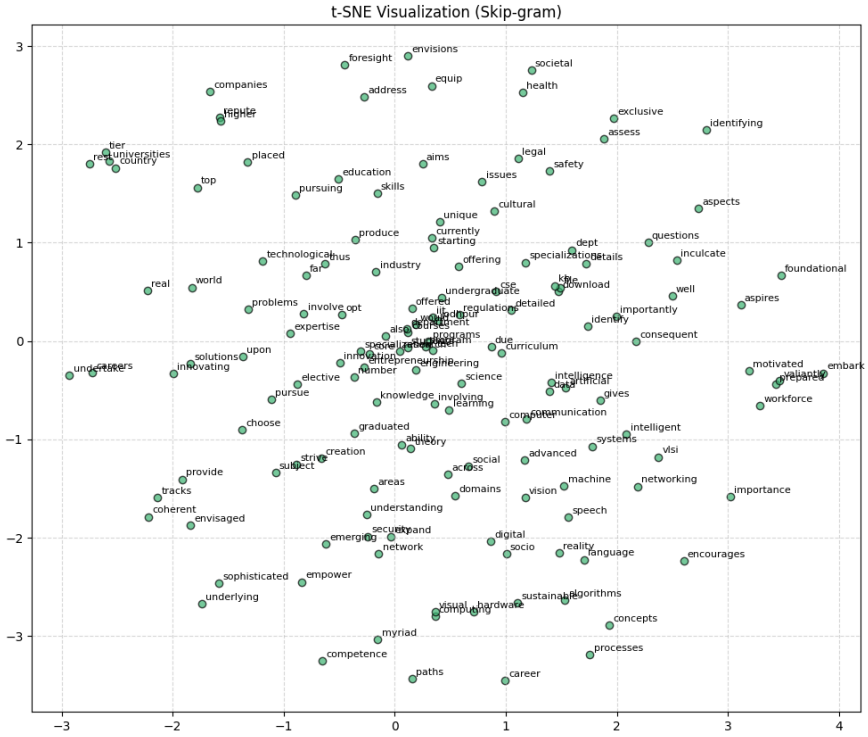
We utilized t-SNE to project the 100-dimensional word embeddings down to a 2D space for visualization.

CBOW Visualization:

The CBOW plot shows a somewhat dispersed arrangement. Because CBOW averages context windows, it tends to group words based on their syntactic roles (e.g., grouping verbs together or administrative filler words together) rather than deep semantic meaning.



Skip-gram Visualization:
The Skip-gram plot shows slight improvements in local clustering for topical words. Because Skip-gram forces the network to predict the surrounding context, semantically related domain words (e.g., academic ranks, course types) are pulled slightly closer together in the high-dimensional space.



General Observation:
In both plots, we do not see massive, highly distinct clusters. This visual sparsity also proves our findings in Task 3: while the model can identify direct nearest neighbors (in Skip-gram), the overall dataset is too small to form the densely packed semantic clusters seen in models trained on data containing billions of words.

Problem 2 - Character level name generation using RNN variants

Task 0 - Dataset Preparation -

A dataset of 1,000 unique Indian names was generated using a LLM. The names were formatted, preprocessed to include start-of-sequence (<SOS>), end-of-sequence (<EOS>), and padding (<PAD>) tokens, and stored in a clean TrainingNames.txt file to serve as the training corpus.

Task 1 - Model Implementation -

Three recurrent neural network architectures were implemented from scratch using PyTorch.
Global Hyperparameters

- Embedding Dimension: 64
- Hidden State Size: 128
- Learning Rate: 0.003
- Epochs run for each model : 50

1. Vanilla Recurrent Neural Network (RNN) -

- At each time step, it takes the current character embedding and the previous hidden state to compute the next hidden state.
- A fully connected linear layer maps this hidden state to the vocabulary logits to predict the next character.
- Trainable Parameters : 30236

2. Bidirectional Long Short-Term Memory (BLSTM) -

- It utilizes two LSTM layers, one processing the sequence forward and the other backward.
- The final hidden states are concatenated before being passed through the fully connected layer.
- Trainable Parameters : 207644

3. RNN with Basic Attention Mechanism

- Builds upon the Vanilla RNN with an additional self-attention mechanism.
- It computes attention scores for all past hidden states, applies a Softmax function to normalize these weights, and calculates a weighted context vector.
- This context vector is then concatenated with the current RNN output to inform the final character prediction.
- Trainable Parameters : 33949

Task 2 - Quantitative Evaluation -

The models were evaluated based on two metrics:

- **Novelty Rate:** The percentage of generated names that do not appear in the original training set.

- **Diversity:** The percentage of unique names within the generated batch.

Model	Novelty Rate	Diversity
Vanilla RNN	24.50%	91.00%
BLSTM	100.00%	20.45%
RNN + Attention	100.00%	4.50%

Observation -

- The Vanilla RNN achieved the healthiest balance, demonstrating strong generalization.
- A 24.50% novelty rate proves it learned the underlying phonotactics of Indian names to invent new ones, while a 91% diversity rate shows it did not get stuck generating the same names repeatedly.
- The BLSTM and RNN+Attention models achieved 100% novelty, but their diversity scores (20.45% and 4.50%, respectively) indicate severe collapse, i.e. they generated highly repetitive, invalid strings that technically counted only because they were not real names.

Task 3 - Qualitative Analysis -

Realism of Generated Names -

Vanilla RNN successfully generated highly realistic and culturally coherent Indian names. Representative samples include perfect matches to real names (eg - Vasant, Gita, etc.) as well as nice novel inventions (eg - Garika).
It clearly learned the alternating consonant-vowel structures shown by the dataset.

Common Failures -

For BLSTM -

The BLSTM generated short, mostly consonant used samples. This happens because a BLSTM is trained by looking at both past and future characters simultaneously. However, during step-by-step generation, the future characters do not exist yet.

For RNN + Attention -

The attention model suffered from an extreme repetition loop, repeating the characters "f", "x", and "z". In small datasets with short sequences, attention mechanisms can easily show this error. It created a broken loop of identical characters rather than learning meaningful phonetic alignments.